

Intelligent Urban Traffic Monitoring System

Project Report

Group Members:

1. Ayan Adnan (DT-22014)
2. Shehroz Ejaz (DT-22011)
3. Aman Qazi (DT-22044)
4. Awais Waseem (DT-22033)

Course Code: CT-361

Course Title: AI&ES

Submission Date: Nov 22, 2024

Project Overview

This project aims to develop an Intelligent Urban Traffic Monitoring System that autonomously monitors, analyzes, and provides insights on traffic flow and incidents across a busy city. By leveraging AI techniques such as object detection, sound classification, and graph-based modeling, the system identifies vehicles, monitors traffic density, and detects incidents in real-time using visual and audio cues. The system will contribute to more efficient urban traffic management and incident response.

Definitions

- **Object Detection:** Object detection is a computer vision task where an algorithm identifies and localizes objects (in this case, vehicles) in an image or video.
- **Sound Analysis:** Sound analysis involves processing audio data to detect certain patterns or events, such as sirens, car honks, or crashes.
- **Graph-Based Traffic Flow and Route Optimization:** Graph-based models represent a city's road network as a graph, where intersections are nodes and roads are edges. This model is used to optimize traffic flow and identify the best routes.

Project Need:

The Intelligent Urban Traffic Monitoring System is crucial for addressing urban traffic challenges, such as congestion, accidents, and inefficient route management. With rapidly growing cities, there is a need for real-time monitoring and optimization to improve traffic flow. By utilizing AI techniques like object detection, sound analysis, and graph-based modeling, the system can provide actionable insights, enhance road safety, and optimize routes, ultimately reducing congestion and improving overall traffic management.

Assumptions

1. The system will use pre-trained models for initial implementation and training datasets tailored for urban environments.
2. Traffic cameras and microphones at intersections provide continuous, high-quality input data.
3. Real-time processing and analysis are feasible with the available computational resources.

Key Tasks

Image and Video Detection

- **Purpose:** Detect and classify different types of vehicles (e.g., cars, buses, trucks, motorcycles) in video streams to monitor traffic density.
- **Implementation:**
 1. **YOLOv5 for Object Detection:**
 - The `detect.py` script from the YOLOv5 repository is used to identify vehicles in the input video.
 - Outputs include labeled bounding boxes for detected vehicles.
 2. **Traffic Density Analysis:**
 - Detected vehicles are counted for each frame.
 - Congestion levels are categorized as:
 - **Low** (fewer than 5 vehicles)
 - **Medium** (5–14 vehicles)
 - **High** (15 or more vehicles)
 - Results are visualized using a line plot of vehicle count over time.
 3. **Output:**
 - Processed video and a CSV file containing frame-wise vehicle counts and congestion levels.

Sound Analysis

- **Purpose:** Identify and classify urban sound patterns to detect incidents (e.g., sirens for emergencies, crashes, or honking indicating blockages).
- **Implementation:**
 1. **Feature Extraction:**
 1. Audio features such as waveform and **Mel-Frequency Cepstral Coefficients (MFCCs)** are extracted using `librosa`.
 2. MFCCs provide a compact representation of audio signals for analysis.
 2. **Deep Learning Classification:**
 3. A **Neural Network** is trained to classify audio events.
 4. Training involves features extracted from the audio dataset.
 5. Output includes predictions of audio events (e.g., "Siren," "Crash") for real-time analysis.
 3. **Output:**
 6. Visualized waveform and MFCC spectrograms.
 7. A trained model capable of classifying urban audio events.

Graph-Based Traffic Flow and Route Optimization

- **Purpose:** Optimize traffic flow using graph-based modeling of the road network.
- **Implementation:**
 1. **Graph Representation:**
 1. Intersections are modeled as **nodes**, and roads as **edges** with attributes:
 - **Length** (in kilometers)
 - **Average Travel Time** (adjusted based on congestion levels)
 - **Congestion Level** (Low, Medium, High)
 2. **Route Optimization Algorithms:**
 2. Algorithms like **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** identify optimal routes between locations.
 3. Travel times are dynamically adjusted based on congestion levels:
 - **High Congestion:** +50% travel time
 - **Medium Congestion:** +20% travel time
 3. **Visualization:**
 4. The road network graph is plotted with nodes (locations) and edges (roads) labeled with congestion-adjusted travel times.

Important terms:

- **!git clone <URL>:**
Used in Jupyter/Colab to clone a GitHub repository onto the local environment.
- **!pip install <package>:**
Installs specified Python libraries or packages directly within the notebook environment.
- **files.upload():**
Uploads files from the local system to the Colab environment for processing.
- **librosa.load(file_path, sr=None):**
Loads an audio file and returns the waveform (y) and sample rate (sr).
- **plt.figure(figsize=(10, 4)):**
Creates a new figure for plotting with a specified size.
- **librosa.feature.mfcc():**
Extracts Mel-Frequency Cepstral Coefficients (MFCC) features from an audio signal.
- **train_test_split(X, y, test_size=0.2, random_state=42):**

Splits data into training and testing subsets, with 20% of the data used for testing.

- **Sequential():**
Initializes a linear stack of layers for building a neural network model in TensorFlow/Keras.
- **model.add(Dense(units, activation='relu')):**
Adds a fully connected (dense) layer to the neural network with a specified number of units and activation function.
- **model.fit(X_train, y_train, epochs=25, batch_size=32, validation_split=0.2):**
Trains the model on the training data for a specified number of epochs and batch size, using part of the data for validation.

Code Snippets:

1. clone a repository from GitHub into the current working directory:

```
!git clone https://github.com/ultralytics/yolov5
```

2. upload files:

```
from google.colab import files
uploaded = files.upload()
```

3. Counting Vehicle Instances from Detection Results:

```
import os
import matplotlib.pyplot as plt

# Folder containing detection results
labels_path = "/content/yolov5/runs/detect/exp/labels"

# Vehicle class IDs (COCO dataset: car=2, bus=5, truck=7, motorcycle=3)
vehicle_classes = {2: "Car", 5: "Bus", 7: "Truck", 3: "Motorcycle"}

# Store frame-wise vehicle counts
frame_counts = []

for file in sorted(os.listdir(labels_path)):
    frame_file = os.path.join(labels_path, file)
    with open(frame_file, 'r') as f:
        lines = f.readlines()
        count = sum(1 for line in lines if int(line.split()[0]) in vehicle_classes)
        frame_counts.append(count)

# Print sample counts
print(f"Sample frame counts: {frame_counts[:10]}")
```

4. Traffic Density Analysis and Visualization Based on Vehicle Counts

```
# Define the categorize_congestion function
def categorize_congestion(vehicle_count):
    if vehicle_count < 5:
        return "Low"
    elif 5 <= vehicle_count < 15:
        return "Medium"
    else:
        return "High"

# Categorize congestion levels
congestion_levels = [categorize_congestion(count) for count in frame_counts]

# Plot traffic density trends
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(range(len(frame_counts)), frame_counts, label="Vehicle Count", color="blue")
plt.axhline(y=5, color="green", linestyle="--", label="Low/Medium Threshold")
plt.axhline(y=15, color="red", linestyle="--", label="Medium/High Threshold")
plt.title("Traffic Density Over Time")
plt.xlabel("Frame")
plt.ylabel("Vehicle Count")
plt.legend()
plt.show()
```

5. BFS and DFS in graph:

```
def bfs_shortest_path(graph, start, goal):
    queue = [(start, [start], 0)] # (current_node, path, total_travel_time)
    visited = set()

    while queue:
        current_node, path, total_time = queue.pop(0)
        if current_node == goal:
            return path, total_time

        visited.add(current_node)

        for neighbor in graph.neighbors(current_node):
            if neighbor not in visited:
                travel_time = graph.edges[current_node, neighbor]["travel_time"]
                queue.append((neighbor, path + [neighbor], total_time + travel_time))

    return None, float("inf") # If no path found

# DFS Algorithm to find a route (based on adjusted travel time)
def dfs_shortest_path(graph, start, goal, visited=None, path=None, total_time=0):
    if visited is None:
        visited = set()
    if path is None:
        path = [start]

    if start == goal:
        return path, total_time

    visited.add(start)

    for neighbor in graph.neighbors(start):
        if neighbor not in visited:
            travel_time = graph.edges[start, neighbor]["travel_time"]
            new_path, new_time = dfs_shortest_path(graph, neighbor, goal, visited, path + [neighbor], total_time + travel_time)
```

6. Audio Signal Analysis:

```

import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt

# Load audio file
file_path = 'police-siren-99029.mp3' # Replace with your file path
y, sr = librosa.load(file_path, sr=None)

# Display waveform
plt.figure(figsize=(10, 4))
librosa.display.waveshow(y, sr=sr)
plt.title('Waveform')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.show()

# Extract MFCC features
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
plt.figure(figsize=(10, 4))
librosa.display.specshow(mfcc, x_axis='time', sr=sr)
plt.colorbar()
plt.title('MFCC')
plt.show()

```

7. Audio Feature Extraction and Labeling for Classification:

```

def get_label(file_name):
    if "collision" in file_name.lower():
        return "minor_collision"
    elif "congestion" in file_name.lower():
        return "heavy_congestion"
    elif "emergency" in file_name.lower():
        return "emergency_vehicle"
    else:
        return "unknown"

# Loop through all audio files in the folder
for file in os.listdir(folder_path):
    if file.endswith('.mp3') or file.endswith('.wav'):
        file_path = os.path.join(folder_path, file)
        y, sr = librosa.load(file_path, sr=None)

        # Extract features
        mfcc = np.mean(librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13), axis=1)
        chroma = np.mean(librosa.feature.chroma_stft(y=y, sr=sr), axis=1)
        spectral_contrast = np.mean(librosa.feature.spectral_contrast(y=y, sr=sr), axis=1)

        # Combine all features
        features = np.concatenate((mfcc, chroma, spectral_contrast))

        # Get Label from file name
        label = get_label(file)

        # Append features and label to the dataset
        data.append(np.append(features, label))

# Convert data into a pandas DataFrame
columns = [f'mfcc_{i}' for i in range(13)] + \
           [f'chroma_{i}' for i in range(12)] + \
           [f'spectral_contrast_{i}' for i in range(7)] + ['label']
df = pd.DataFrame(data, columns=columns)

```

8. Data Preprocessing:

```

# Encode Labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)

```

9. Accuracy:

```

# Evaluate on the test set
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=2)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

1/1 - 0s - 18ms/step - accuracy: 1.0000 - loss: 0.9488
Test Accuracy: 100.00%

import matplotlib.pyplot as plt

# Plot accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Model Accuracy')
plt.show()

# Plot Loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Model Loss')
plt.show()

```

10. Predictions:

```

# Make predictions
predictions = model.predict(X_test)
predicted_labels = np.argmax(predictions, axis=1)

# Decode Labels to their original class names
decoded_labels = label_encoder.inverse_transform(predicted_labels)

# Print some predictions

for i in range(len(decoded_labels)): # or len(y_test)
    print(f"Predicted: {decoded_labels[i]}, Actual: {label_encoder.inverse_transform([y_test[i]])[0]}")
print("Number of test samples:", len(y_test))
print("Number of predicted labels:", len(decoded_labels))

```

